

Selezione Dinamica della Dimensione del Campione

in Metodi di Ottimizzazione per il Machine Learning

Questa tesi presenta la metodologia per la selezione dinamica della dimensione del campione negli algoritmi di ottimizzazione per il machine learning su larga scala, fondandosi sul lavoro di Byrd, Chin, Nocedal e Wu (2012). L'idea centrale è che, invece di fissare a priori la dimensione del batch, si possa decidere durante l'algoritmo se e quando aumentarla, guidandosi da stime statistiche della varianza del gradiente stocastico. Il lavoro offre un'analisi teorica completa della convergenza, il miglioramento di alcuni risultati mediante disuguaglianze più strette e un'implementazione Python dei quattro algoritmi principali: gradiente dinamico, Newton-CG, estensione L_1 e BB-CCV, con un'applicazione web interattiva per la sperimentazione.

Agenda

- Contesto: l'ottimizzazione su larga scala e il mini-batch
- L'idea: campionamento dinamico e Condizione di Controllo della Varianza
- I quattro algoritmi (Dynamic GD, Newton-CG, Newton-CG L_1 , BB-CCV)
- Risultati teorici di convergenza e complessità
- App web interattiva e setup sperimentale
- Esperimenti numerici e applicazioni su NSynth
- Riutilizzo del mini-batch, conclusioni e lavoro futuro

Contesto: ottimizzazione su larga scala

- ML su larga scala: dataset con $N \approx 10^6$ – 10^9 esempi, perdita media $J(w)$
- Gradiente completo $\nabla J(w)$: costo $O(Nm)$ a iterazione, proibitivo
- Standard: metodi stocastici su batch (SGD, metodi a varianza ridotta)
- Il batch è un sottoinsieme dei dati usato a ogni passo
- Riferimento: Byrd, Chin, Nocedal e Wu (2012)

Il problema: la dimensione del mini-batch

- Batch piccolo → iterazioni economiche, ma gradiente rumoroso
 - il rumore limita la precisione raggiungibile
- Batch grande → stima accurata, ma costo proporzionale alla dimensione
- Nei metodi classici la dimensione è fissa e scelta in modo euristico
- Troppo piccolo: non si converge alla precisione desiderata
- Troppo grande: risorse sprecate nelle prime iterazioni

L'idea: campionamento dinamico

- Si parte da un batch piccolo: progressi rapidi ed economici
- Lo si aumenta progressivamente quando serve precisione
- La decisione è guidata dalla Condizione di Controllo della Varianza (CCV)
- Risorse concentrate dove servono davvero, con adattamento automatico
 - alla difficoltà del problema
- Nessuna scelta euristica a priori del batch

La Condizione di Controllo della Varianza (CCV)

- Finché la varianza campionaria del gradiente stocastico è piccola rispetto alla norma del gradiente, il batch è adeguato:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|\nabla J(w_k)\|_2^2$$

- Vicino all'ottimo il gradiente tende a zero: il rumore relativo cresce e la CCV impone un batch più grande
- Il batch cresce a “scalini”, esattamente quando la soglia viene superata
- θ piccolo $\rightarrow$ batch cresce presto; θ grande $\rightarrow$ batch piccolo più a lungo

I quattro algoritmi

- Dynamic GD: gradiente a campione dinamico (base del metodo)
- Newton-CG: Hessiana sottocampionata ($|\mathcal{H}| = R |\mathcal{S}|$) e criterio di arresto adattivo del CG interno
- Newton-CG L_1 : gradiente generalizzato, identificazione dell'active set e line search proiettata
- BB-CCV: passo di Barzilai–Borwein con campionamento dinamico
- Tutti implementati in Python (Appendice B)

Line search: Armijo e Wolfe

Dalla tesi — Appendice A, condizioni sul passo α

Armijo (limite superiore su f): il passo α è accettato se

$$f(x_k + \alpha p) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p,$$

con $c_1 \in (0, 1)$ (tipicamente 10^{-4}): la retta $f(x_k) + c_1 \alpha \nabla f(x_k)^T p$ funge da upper bound; i passi che la superano vengono rifiutati e ridotti.

Curvatura di Wolfe (limite inferiore su g'), con $g(\alpha) = f(x_k + \alpha p)$ e $g'(\alpha) = \nabla f(x_k + \alpha p)^T p$:

$$g'(\alpha) \geq c_2 g'(0), \quad c_2 \in (c_1, 1), \quad g'(0) = \nabla f(x_k)^T p < 0.$$

La condizione $0 < c_1 < c_2 < 1$ garantisce un intervallo non vuoto di passi accettabili: Armijo scarta i passi troppo grandi, Wolfe quelli troppo piccoli.

La Condizione di Controllo della Varianza (CCV)

Dalla tesi — Sez. 5.1, Metodo del Gradiente a Campione Dinamico

Varianza campionaria del gradiente stimato sul batch $\mathcal{S}_k$:

$$\hat{\mathcal{V}} = \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in \mathcal{S}_k} \left(\nabla \ell(w_k; i) - \nabla J_{\mathcal{S}_k}(w_k) \right)^2.$$

La varianza stimata deve essere piccola rispetto al gradiente corrente:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2, \quad \theta \in (0, 1).$$

Se la CCV è violata, la nuova dimensione del batch è

$$n_k^{\text{new}} = \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2} \right\rceil.$$

La CCV garantisce una direzione di discesa

Dalla tesi — Sez. 5.1, condizione di discesa e Lemma di accuratezza

$-g_k$ è di discesa per J se l'errore relativo del gradiente stimato è piccolo:

$$\|e_k\|_2 \leq \theta \|g_k\|_2, \quad e_k := g_k - \nabla J(w_k), \quad \theta \in [0, 1).$$

Infatti, ponendo $g_k = \nabla J(w_k) + e_k$ e usando Cauchy-Schwarz ($|e_k^T g_k| \leq \|e_k\| \|g_k\|$):

$$\nabla J(w_k)^T g_k \geq \|g_k\|^2 - \|e_k\| \|g_k\| = \|g_k\| (\|g_k\| - \|e_k\|) > 0 \quad \implies \quad -g_k \text{ scende.}$$

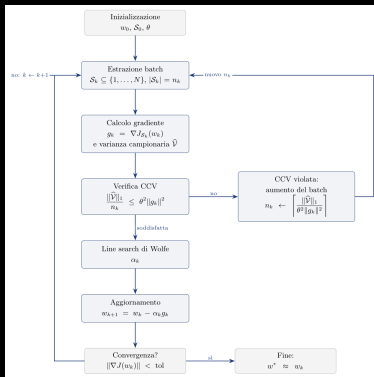
Elevando al quadrato, $\|e_k\|_2^2 \leq \theta^2 \|g_k\|_2^2$; e poiché $\mathbb{E}[g_k] = \nabla J(w_k)$:

$$\mathbb{E}[\|e_k\|_2^2] = \mathbb{E}[\|g_k - \mathbb{E}[g_k]\|_2^2] = \|\text{Var}(g_k)\|_1.$$

Stimando $\|\text{Var}(g_k)\|_1 \approx \|\hat{\mathcal{V}}\|_1/n_k$, la disuguaglianza in attesa diventa esattamente la CCV:

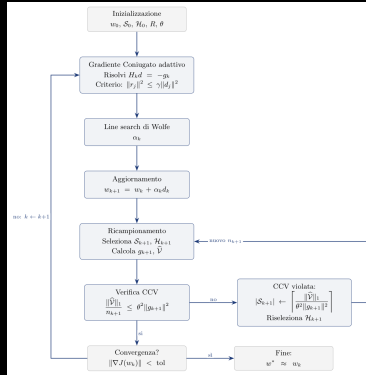
$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2.$$

Schema: Dynamic GD (Fig. 5.1)



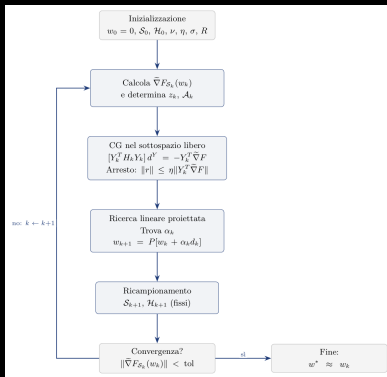
Schema dell'algoritmo con controllo della varianza campionaria (CCV)

Schema: Newton-CG (Fig. 5.2)



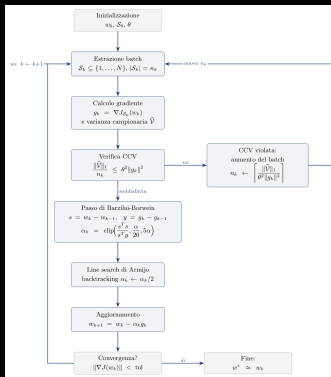
Schema del metodo Newton-CG con campionamento dinamico

Schema: Newton-CG L_1 (Fig. 5.5)



Schema del metodo Newton-CG per problemi con regolarizzazione L_1

Schema: BB-CCV (Fig. 5.6)



Schema dell'algoritmo BB-CCV (gradiente dinamico con passo Barzilai–Borwein)

Risultati teorici

- Sotto la CCV la direzione di ricerca è di discesa e il metodo converge linearmente (forma deterministica e in aspettativa)
- Complessità totale: $O(mL/(\lambda\varepsilon))$ — competitiva con SGD
- Su problemi mal condizionati il termine κ^2 di SGD diventa κ
- Fattore di contrazione migliorato: da $1 - \frac{(1-\theta)\lambda}{2L}$ a $1 - \frac{(1-\theta)\lambda}{L}$ usando la disuguaglianza di Polyak–Łojasiewicz in forma forte

App web interattiva

- `visualizzazione.html`: Pyodide + Plotly, nessuna installazione
- Codice Python dei 4 algoritmi visibile e modificabile nel browser
- Preset quadratici + dataset sintetico “centrato” ($\hat{J}_N = J$)
- Percorso 3D/2D su $J(w)$, metriche in tempo reale, pannelli “Analisi” e “Test batch”
- Permette di osservare la dinamica del batch e l’effetto di θ

Setup e risultati numerici

- Setup: $N = 200$, seed 42, $w_0 = (2, -3)$, $\alpha = 0.1$, $\theta = 0.5$, $n_0 = 5$, 30 iterazioni, $\|\nabla J\| < 10^{-6}$
- Preset: ben condizionato ($\kappa \approx 1.1$), mal condizionato ($\kappa \approx 20$), molto mal condizionato ($\kappa \approx 100$), incrociato
- Profilo a “scalini” del batch confermato dagli esperimenti
- BB-CCV raggiunge la precisione macchina ($\approx 1.1 \times 10^{-14}$) su $\kappa \approx 100$

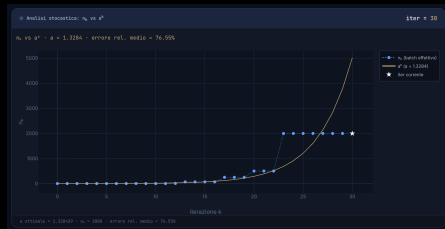


Fig. 5.3 — dimensione del campione n_k vs iterazioni

Esperimenti NSynth: riconoscimento della nota

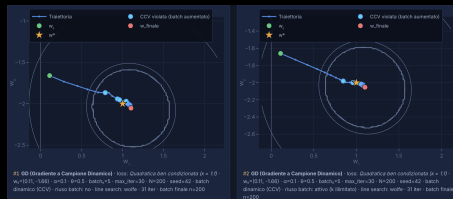
- Dataset audio NSynth: classificazione delle 12 note
- Newton-CG con campionamento dinamico è competitivo con L-BFGS (91.4% vs 91.5%)
- L_1 produce modelli sparsi: 146 feature attive
- Batch finale 12 678 (intero validation set)

Esperimenti NSynth: rete neurale

- Rete neurale: validation set di 12 678 clip
- Newton-CG è il migliore (99.2%): sfrutta la curvatura dell'Hessiana sottocampionata
- Il batch cresce con la CCV ($n_0 = 128 \rightarrow 2\,048$)
- L_1 : modello più semplice, solo 58 412 parametri attivi

Riuso del mini-batch (Sez. 6.7)

- Variante: riusare lo stesso campione per M iterazioni consecutive
- Si riusa la lista degli indici, NON il gradiente
 - ricalcolato a ogni iterazione al punto corrente
- La CCV fa da segnale di esaurimento: se non è soddisfatta, n_k cresce e si ricampiona
- $M = \infty$: si tiene il campione finché la CCV vale
- L'effetto dipende dal metodo e dal condizionamento del problema



Traiettorie con riuso del batch

Conclusioni

- Analisi teorica completa del campionamento dinamico: convergenza lineare (deterministica e in aspettativa), complessità
- Migliorati i risultati di convergenza (PL forte, fattore di contrazione)
- Implementazione Python dei 4 algoritmi + app web interattiva
- Esperimenti coerenti con la teoria; su NSynth Newton-CG è competitivo con L-BFGS (nota) e raggiunge il 99.2% (rete)
- Riuso del mini-batch: meccanismo analizzato con effetti dipendenti da metodo e condizionamento

Limiti e lavoro futuro

- Limiti: ipotesi di convessità forte, varianza limitata, dataset sintetico centrato
- Futuro: assumere direttamente la condizione di Polyak–Łojasiewicz (copre funzioni non convesse)
- Futuro: rapporto di sottocampionamento R dell'Hessiana dinamico